Parte 3. Elegir y adaptar las estructuras

Solicitudes y deshacer

Integrante A **Anderson Gil Arenas**

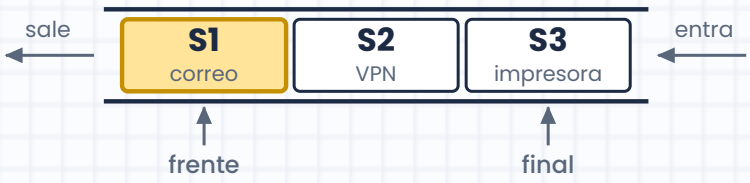
Propuso, dibujó y verificó: **Anderson Gil Arenas**

3.1 Solicitudes: cola

Elección: cola (FIFO). Las solicitudes se atienden en el orden en que llegan, que es exactamente la regla de una cola: la primera en entrar es la primera en salir. Cada solicitud nueva se agrega al final con `enqueue` y la siguiente por atender se toma del frente con `dequeue`; ninguna operación necesita recorrer la estructura. Una pila no sirve porque atendería primero a la última que llegó.

Llegan en orden S1, S2 y S3. **Se atiende primero S1**, que está en el frente.

S cola de solicitudes



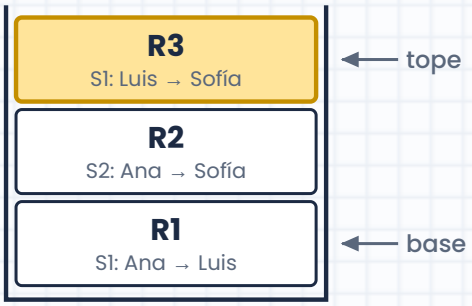

```
enqueue(S1)   // 08:02 correo
enqueue(S2)   // 08:05 VPN
enqueue(S3)   // 08:09 impresora
dequeue()     // → S1, se atiende
```

3.2 Deshacer reasignaciones: pila

Elección: pila (LIFO). Deshacer debe revertir primero la reasignación más reciente, que es la última que entró: justo lo que devuelve `pop`. Cada reasignación se apila con `push` guardando qué solicitud se movió, de qué técnico y a cuál, para poder devolverla. Una cola no sirve porque desharía primero la más antigua.

Se hicieron R1, R2 y R3 en ese orden. **Se deshace primero R3**, que está en el tope: S1 vuelve de Sofía a Luis.

D pila de deshacer




```
push(R1)   // S1: Ana → Luis
push(R2)   // S2: Ana → Sofía
push(R3)   // S1: Luis → Sofía
pop()      // → R3: S1 vuelve a Luis
pop()      // → R2: S2 vuelve a Ana
pop()      // → R1: S1 vuelve a Ana
```

Parte 3. Elegir y adaptar las estructuras

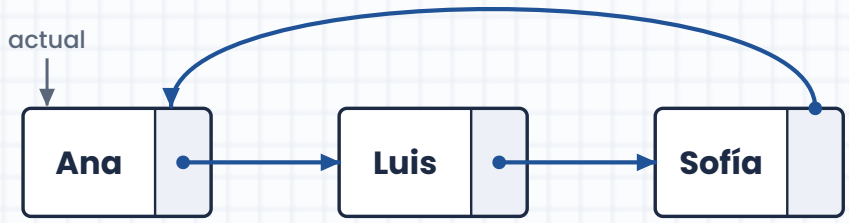
Técnicos: estructura antes y después del cambio

3.3 Turnos de técnicos: de circular simple a doble circular

Momento	Estructura	Qué guarda cada nodo	Operaciones
Antes del cambio	**Lista circular simple**	técnico y siguiente	avanzar: `actual ← actual.siguiente`
Después del cambio	**Lista doble circular**	técnico, siguiente y **anterior**	avanzar: `actual ← actual.siguiente` retroceder: `actual ← actual.anterior`

Antes. Los turnos se repiten y al terminar el último se vuelve al primero: en una lista circular el siguiente del último nodo es el primero, así que avanzar nunca llega a nulo y no hace falta ningún caso especial. Como al principio solo se avanza, basta un enlace por nodo.

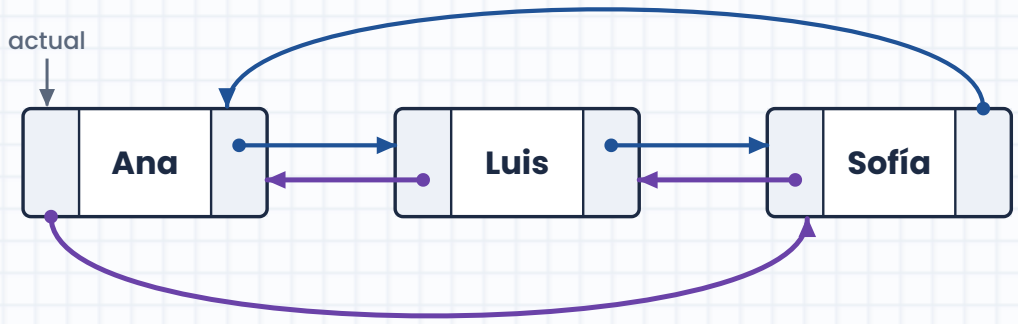
Antes: circular simple. Sofía.siguiente = Ana cierra el ciclo.



Después. Ahora también hay que pasar al técnico anterior, incluso del primero al último. En la circular simple eso obligaría a dar n − 1 pasos hacia adelante. Se agrega a cada nodo la referencia **anterior**, y como la lista sigue siendo circular, el anterior del primero es el último: Ana.anterior = Sofía. Así retroceder es un solo paso, igual que avanzar.



Versión final: doble circular con todos sus enlaces.



Nodo	siguiente	anterior (nuevo)
Ana	Luis	Sofía
Luis	Sofía	Ana
Sofía	Ana	Luis

Información adicional por nodo: una referencia **anterior** al técnico previo en el turno. Se debe mantener la simetría `x.siguiente.anterior = x` y `x.anterior.siguiente = x`; al agregar o retirar un técnico se actualizan cuatro referencias en lugar de dos.

Alternativa aceptable para la versión inicial: una cola rotada con dequeue y enqueue, pero tampoco permite retroceder en un paso.